

Module 1: Introduction to Databases & RDBMS Fundamentals

Q1. Two file-system problems

The two problems are data redundancy and data inconsistency. The same customer address is stored in multiple places, so it is duplicated. If the customer changes the address in only one spreadsheet, the copies can disagree; a database provides a central controlled record.

Q2. Three things a DBMS does that a file system does not

1. Concurrency control — prevents simultaneous banking updates from producing incorrect balances.
2. Constraints/integrity enforcement — prevents invalid or unrelated records from being stored.
3. Transactions and recovery — prevents failures from leaving operations only partially completed.

Q3. Relational terminology

Relation → Table → customer

Tuple → Row/record → One customer such as customer 1001

Attribute → Column/field → first_name

Degree → Number of columns → Number of columns in customer

Cardinality → Number of rows → Number of rows in customer

Q4. Keys

1. customer.customer_id — Primary key
2. account.account_number — Candidate key
3. account.customer_id — Foreign key
4. account.balance — None
5. employee.manager_id — Foreign key
6. branch.ifsc_code — Candidate key

Q5. OLTP or OLAP

1. Customer checks balance — OLTP — Real-time operational query.
2. Risk calculates NPLs by branch for the quarter — OLAP — Historical analytical

aggregation.

3. ATM dispenses ₹10,000 — OLTP — Real-time transaction.

4. Regulator asks for transaction total above ₹200,000 for last financial year — OLAP — Historical aggregation.

5. Teller opens a savings account — OLTP — Real-time operational insert/update.

6. Marketing finds top 100 customers by average monthly balance — OLAP — Analytical aggregation.

Q6. Why not run regulatory reports directly against the core Oracle database?

Large analytical queries can consume resources needed by real-time transactions, long-running reports can cause contention, and repeated scans can reduce OLTP performance. Use a separate reporting/data-warehouse or analytical system populated through controlled data pipelines.

Q7. ACID matching

Atomicity → d. Money is debited but never credited

Consistency → c. An account exists for a customer who does not

Isolation → b. Two ATMs both pay out against the same balance

Durability → a. A committed transfer disappears after a power cut

Q8. Write-ahead log

The write-ahead log is written to disk before the modified data page so recovery has a durable record of the change. If the data page were written first and a crash occurred before the log was written, recovery might not have the information needed to correctly recover the change.

Q9. SELECT * and data independence

It breaks logical data independence. Adding a `pan_number` column changes the result shape returned by `SELECT *`, which can break application code that assumes a fixed number or order of columns.

Q10. Oracle, PostgreSQL, MySQL and Redis

Oracle — Core enterprise banking workloads.

PostgreSQL — Applications requiring a powerful open-source relational database.

MySQL — Web/application workloads where a relational database is appropriate.

Redis — Caching, sessions and other low-latency in-memory use cases.

Q11. Database name, current user and server port

```
SELECT current_database(), current_user, inet_server_port();
```

Q12. Every column of loan

```
SELECT ordinal_position, column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'public' AND table_name = 'loan'
ORDER BY ordinal_position;
```

```
SELECT COUNT(*) AS column_count
FROM information_schema.columns
WHERE table_schema = 'public' AND table_name = 'loan';
```

The column count is the degree of the loan relation.

Q13. Row counts of all nine tables

```
SELECT 'account' AS table_name, COUNT(*) AS row_count FROM account
UNION ALL SELECT 'account_txn', COUNT(*) FROM account_txn
UNION ALL SELECT 'beneficiary', COUNT(*) FROM beneficiary
UNION ALL SELECT 'branch', COUNT(*) FROM branch
UNION ALL SELECT 'card', COUNT(*) FROM card
UNION ALL SELECT 'customer', COUNT(*) FROM customer
UNION ALL SELECT 'employee', COUNT(*) FROM employee
UNION ALL SELECT 'loan', COUNT(*) FROM loan
UNION ALL SELECT 'loan_payment', COUNT(*) FROM loan_payment
ORDER BY table_name;
```

Q14. Foreign keys

```
SELECT tc.table_name AS child_table, kcu.column_name AS child_column, ccu.table_name AS
parent_table, ccu.column_name AS parent_column
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu ON tc.constraint_name =
kcu.constraint_name AND tc.table_schema = kcu.table_schema
JOIN information_schema.constraint_column_usage AS ccu ON tc.constraint_name =
ccu.constraint_name AND tc.table_schema = ccu.table_schema
WHERE tc.constraint_type = 'FOREIGN KEY' AND tc.table_schema = 'public'
ORDER BY child_table, child_column;
```

The self-referencing foreign key is employee.manager_id → employee.employee_id, representing the employee managerial hierarchy.

Q15. Three largest tables

```
SELECT relname AS table_name, pg_size_pretty(pg_total_relation_size(relid)) AS
total_size
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 3;
```

account_txn is the largest because it stores individual account transactions and has substantially more records than the other operational tables; indexes also contribute to its total size.

Q16. Insert Latha Krishnan and account

```
BEGIN;
```

```
INSERT INTO customer (customer_id, first_name, last_name, date_of_birth, email, phone,
city, kyc_status, risk_rating, customer_since, home_branch)
VALUES (1032, 'Latha', 'Krishnan', '1985-09-23', 'latha.krishnan@example.com',
'9845099887', 'Chennai', 'VERIFIED', 'MEDIUM', '2026-03-01', 5);
```

```

INSERT INTO account (account_id, account_number, customer_id, account_type, balance,
opened_date, interest_rate)
VALUES (9047, 'ICIC000009047', 1032, 'CURRENT', 50000.00, '2026-03-01', 0.00);

SELECT * FROM customer WHERE customer_id = 1032;
SELECT * FROM account WHERE account_id = 9047;

ROLLBACK;

```

Q17. Deposit ₹75,000 and withdraw ₹20,000

```

BEGIN;
UPDATE account SET balance = balance + 75000 WHERE account_id = 9047;
UPDATE account SET balance = balance - 20000 WHERE account_id = 9047;
SELECT account_id, account_number, balance FROM account WHERE account_id = 9047;
ROLLBACK;

```

Q18. Delete customer in the wrong order

```

BEGIN;
DELETE FROM customer WHERE customer_id = 1032;
-- This should fail because account 9047 references customer 1032.
-- Record the exact PostgreSQL error returned by your database.
DELETE FROM account WHERE account_id = 9047;
DELETE FROM customer WHERE customer_id = 1032;
SELECT COUNT(*) FROM customer;
SELECT COUNT(*) FROM account;
ROLLBACK;

```

Q19. Three statements the database must reject

1. Invalid account type: INSERT an account with account_type = 'FIXED'.
 2. Negative transaction amount: insert -5000 into the amount column using the table's required columns.
 3. Invalid KYC status: INSERT a customer with kyc_status = 'IN_PROGRESS'.
- Record the exact PostgreSQL error returned for each failed statement.

Q20. Two-session isolation

Session A:

```

BEGIN;
UPDATE account SET balance = balance + 1000 WHERE account_id = 9004;
SELECT account_id, balance FROM account WHERE account_id = 9004;

```

Do not commit.

Session B:

```

SELECT account_id, balance FROM account WHERE account_id = 9004;

```

Session B should not see Session A's uncommitted update.

Session A:

```

ROLLBACK;

```

Q21. Transfer ₹12,000 and roll it back

```
SELECT account_id, balance FROM account WHERE account_id IN (9004, 9010) ORDER BY account_id;
```

```
BEGIN;
```

```
UPDATE account SET balance = balance - 12000 WHERE account_id = 9004;
```

```
UPDATE account SET balance = balance + 12000 WHERE account_id = 9010;
```

```
SELECT account_id, balance FROM account WHERE account_id IN (9004, 9010) ORDER BY account_id;
```

```
ROLLBACK;
```

```
SELECT account_id, balance FROM account WHERE account_id IN (9004, 9010) ORDER BY account_id;
```

Q22. PostgreSQL configuration

```
SHOW shared_buffers;
```

```
SHOW work_mem;
```

```
SHOW wal_level;
```

```
SHOW fsync;
```

shared_buffers — shared data-page caching memory.

work_mem — memory available for operations such as sorting and hashing.

wal_level — amount/detail of information written to the WAL.

fsync — controls forcing data/WAL writes to durable storage.

Q23. MyISAM for a 4,000 TPS UPI system

I would not use MyISAM for this core transaction workload because it does not provide the same transaction and crash-recovery guarantees required for financial transfers. I would use InnoDB, which provides transactional behavior, row-level locking and crash recovery.

Q24. 40 million account_txn rows

One system optimized for real-time account lookups is not necessarily best for large analytical aggregations. Use the operational database for branch/account operations and a separate analytical/data-warehouse system for reporting and risk analysis.

Q25. 'Transfer successful' in application log

An application log message only proves that the application wrote that message; it does not prove that the database transaction committed. Evidence would be a successful database commit with persisted account balances/records, verified from the database or transaction/audit records.

Q26. fsync = off

I would not set fsync = off on the core banking PostgreSQL server because it increases the risk of data corruption or loss after a system failure. If used at all, it should be limited to a disposable development/test environment where data loss is acceptable.

Q27. Why keep balance ≥ 0 in the database?

1. The database is shared by multiple applications, so central enforcement prevents one application from bypassing the rule.
2. A database constraint protects data even when an application has a bug or a new application is introduced.

The application layer is appropriate when a rule depends on complex external context or user interaction that cannot reasonably be expressed as a database constraint.

Q28. PostgreSQL vs Oracle differences

1. SQL dialect and syntax can differ.
2. Data types can have different behavior and limits.
3. Sequences/identity mechanisms can differ.
4. Database architecture and administration, including database/schema/connection concepts and configuration, differ.

Q29. Ticket ICICI-4417

Moving from spreadsheets to a relational database will give the bank one controlled place for customer and account information instead of multiple independent copies. When information changes, authorized users can update the central record so departments do not work from conflicting versions. The database can enforce rules such as unique customer identifiers, valid account references and acceptable values. It can control which users can read or change information. Banking operations can be handled as a single unit, so a failure does not leave only half of an operation completed. The database can also recover committed information after a system failure. However, moving to a database will not automatically correct bad data, prevent every application bug, or replace good procedures. The bank must define who owns each piece of information and who may change it. Staff must use approved applications instead of unofficial spreadsheets. The bank should also monitor access, maintain backups, test recovery procedures and review data-quality rules regularly.